

RGB-Only Gaussian Splatting SLAM for Unbounded Outdoor Scenes

Sicheng Yu^{1*}, Chong Cheng^{1*}, Yifan Zhou¹, Xiaojun Yang¹, Hao Wang^{1†}

Abstract—3D Gaussian Splatting (3DGS) has become a popular solution in SLAM, as it can produce high-fidelity novel views. However, previous GS-based methods primarily target indoor scenes and rely on RGB-D sensors or pre-trained depth estimation models, hence underperforming in outdoor scenarios. To address this issue, we propose a RGB-only gaussian splatting SLAM method for unbounded outdoor scenes—OpenGS-SLAM. Technically, we first employ a pointmap regression network to generate consistent pointmaps between frames for pose estimation. Compared to commonly used depth maps, pointmaps include spatial relationships and scene geometry across multiple views, enabling robust camera pose estimation. Then, we propose integrating the estimated camera poses with 3DGS rendering as an end-to-end differentiable pipeline. Our method achieves simultaneous optimization of camera poses and 3DGS scene parameters, significantly enhancing system tracking accuracy. Specifically, we also design an adaptive scale mapper for the pointmap regression network, which provides more accurate pointmap mapping to the 3DGS map representation. Our experiments on the Waymo dataset demonstrate that OpenGS-SLAM reduces tracking error to 9.8% of previous 3DGS methods, and achieves state-of-the-art results in novel view synthesis. Project page: <https://3dagentworld.github.io/opengs-slam/>.

I. INTRODUCTION

Simultaneous Localization and Mapping (SLAM) is a core task in the field of computer vision, extensively applied in autonomous driving, robotics, and virtual reality (VR) [1], [2]. The issue of 3D representation has been a major focus, with the long-term goal of achieving both high-fidelity visual effects and precise localization capabilities [3], [4].

Previous works can be categorized into two branches, one is dense representation based methods [5], [6], [7], another is neural implicit representation based methods [8], [9], [10], [11], [12], [13]. Although the former renders observed regions effectively, they fall short in novel view synthesis capability. The latter approach develops end-to-end differentiable dense visual SLAM systems, presenting strong performance. However, they have limitations such as low computational efficiency and lack of explicit modeling pose.

Recently, there are studies attempting to employ 3D Gaussian Splatting (3DGS) [14] for scene representation [3], [15], [16], [17], which not only enables high-fidelity novel view synthesis but also achieves real-time rendering with lower memory requirements. Existing studies either rely heavily on high-quality depth inputs or only work on the scenarios of small-scale indoor scenes with limited camera movement [3], [4]. Using RGB-only data for unbounded outdoor scenes

remains challenging, due to: 1) difficulties in accurate depth and scale estimation, which impact pose accuracy and 3DGS initialization; 2) limited image overlap and singular viewing angles that lack effective constraints, leading to difficulties in training convergence.

To address the challenges above, this paper proposes a novel 3DGS-based SLAM method for unbounded outdoor scenes, **OpenGS-SLAM**. Our method only adopts RGB information, using 3DGS to represent the scene and generate high-fidelity images.

Specifically, we employ a pointmap regression network to generate consistent pointmaps between frames. These pointmaps store 3D structures from multiple standard views, which contain viewpoint relationships, 2D-to-3D correspondences, and scene geometry. This enables more robust camera pose estimation, effectively alleviating the inaccuracy issues of pre-trained depth networks [18].

Furthermore, we integrate camera pose estimation with 3DGS rendering into an end-to-end differentiable pipeline. By this way, we achieve joint optimization of pose and 3DGS parameters, significantly enhancing system tracking accuracy. We also design an adaptive scale mapper and a dynamic learning rate adjustment strategy, which more accurately maps pointmap to the 3DGS map representation.

Notably, our experiments on the Waymo dataset demonstrate OpenGS-SLAM reduces tracking error to **9.8%** of the existing 3DGS method [3]. We also establish a new benchmark in novel view synthesis, achieving state-of-the-art results.

Our main contributions include:

- 1) To the best of our knowledge, we are the first to propose a RGB-only 3D Gaussian Splatting SLAM method for unbounded outdoor scenes.
- 2) We propose a system integrating a pointmap regression network with an end-to-end pipeline from pose estimation to 3DGS rendering. This allows for simultaneous optimization of pose and scene parameters, significantly improving tracking accuracy and stability.
- 3) With the proposed adaptive scale mapper and dynamic learning rate adjustment, our OpenGS-SLAM achieves state-of-the-art performance in novel view synthesis on the Waymo dataset.

II. RELATED WORK

A. Differentiable Rendering SLAM.

Since NeRF [8] is proposed, many NeRF-based SLAM methods have emerged. iMAP [9] innovatively introduces NeRF as a scene representation in SLAM, utilizing a dual-threaded approach to perform camera pose tracking and

* Authors contributed equally to this work.

¹ The Hong Kong University of Science and Technology (GuangZhou).

† Corresponding author. haowang@hkust-gz.edu.cn

scene mapping simultaneously. NICE-SLAM [10] incorporates hierarchical scene representation to fuse multi-level local information. Vox-Fusion [11] combines traditional volumetric fusion methods with neural implicit representations. ESLAM [12] implements multi-scale axis-aligned feature planes, diverging from traditional voxel grids, significantly improving frame processing speed. Subsequent works, such as GO-SLAM [13], HI-SLAM [19], and Loopy-SLAM [20], incorporate loop closure and global bundle adjustment (BA) into NeRF-based SLAM.

Recently, due to the fast and differentiable rendering capabilities of 3DGS [14], as well as its strong scene representation, some works have begun exploring Gaussian-based SLAM. SplatAM [15] integrates Gaussian scene representation into the SLAM process, optimizing camera pose and Gaussian map by minimizing rendering photometric and depth losses. MonoGS [3] derives the pose Jacobian matrix for tracking optimization and introduced isotropic regularization to ensure geometric consistency. Photo-SLAM [16] combines ORB-SLAM3 [21] with Gaussian scene representation. GS-SLAM [17] proposes a robust coarse-to-fine camera tracking technique to improve tracking speed and accuracy. Gaussian-SLAM [22] introduces submap-based Gaussian scene representation, while CG-SLAM [23] uses a novel uncertainty-aware 3D Gaussian field for consistent and stable tracking and mapping.

B. RGB-only Dense Visual SLAM.

Despite the success of these methods with RGB-D inputs, RGB-only SLAM presents unique challenges, primarily due to the lack of direct depth information which complicates geometric reconstruction. However, the increased difficulty makes RGB-only dense SLAM research more valuable. NeRF-SLAM [24] and Orbeez-SLAM [25] utilize DROID-SLAM [26] and ORB-SLAM2 [27] as tracking modules, respectively, both leveraging Instant-NGP [28] for volumetric neural radiance field mapping. DIM-SLAM [29] and NICER-SLAM [1] perform tracking and mapping on the same neural implicit map represented by hierarchical feature grids, but do not address global map consistency, such as loop closure. GO-SLAM [13] and HI-SLAM [19] extend DROID-SLAM [26] to the full SLAM setting by introducing online loop closure via factor graph optimization. GIORIE-SLAM [30] employs a flexible neural point cloud representation and introduces a novel DSPO layer for bundle adjustment, optimizing keyframe poses and depth.

Recently, some works have started using 3DGS to address the challenges of RGB-only SLAM. MonoGS [3] and Photo-SLAM [16] both support RGB-only inputs and achieve performance comparable to that of RGB-D inputs. MotionGS [31] implements tracking through feature extraction and a motion filter on each frame, using compressed 3D Gaussian representation to reduce memory usage. MGS-SLAM [32] adopts DPVO [33] as tracking module and utilizes a pre-trained MVS network to estimate prior depth, adjusting its scale for Gaussian scene reconstruction. Splat-SLAM [4] combines GIORIE-SLAM [30] with Gaussian scene repre-

sentation, introducing global BA into Gaussian-based SLAM. While substantial progress has been made, particularly in adapting SLAM technologies for indoor environments, the extension to outdoor settings remains limited. The development of robust RGB-only SLAM systems that can handle the unbounded and dynamic nature of outdoor environments is an ongoing area of research, with potential breakthroughs likely to have a significant impact on the field.

III. METHOD

A. SLAM System Overview

Fig. 1 provides an overview of our system. In this section, we introduce our system from the following aspects: Tracking, 3DGS scene representation, and Mapping. Our proposed method is specifically designed to address the challenges of unbounded outdoor scenes, enhancing the tracking accuracy and robustness of scene reconstruction.

B. Tracking

1) *Pairwise Pointmap Regression and Pose Estimation:* Previous works [3], [10] involving 3DGS and NeRF primarily focus on indoor and small-scale scenes where camera movements are minimal and viewing angles are dense. In this scenario, NeRF or 3DGS can be directly used to regress camera poses. However, outdoor scenes typically involve vehicle-based photography, characterized by significant movement amplitudes and relatively sparse viewing angles. This makes direct regression of camera poses exceedingly challenging.

Given that pointmaps contain the viewpoint relationships, 2D-to-3D correspondences, and scene geometry [18], [34], [35], we propose a novel pose estimation method based on a pairwise pointmap regression network, aimed at robust and rapid camera pose estimation for the current frame.

The specific methodology is as follows: Assuming two input 2D images $I^1, I^2 \in \mathbb{R}^{W \times H}$, we define the pointmap as the 3D points $X^1, X^2 \in \mathbb{R}^{W \times H \times 3}$ corresponding to each pixel of these images. We utilize a pre-trained pointmap regression network that combines a ViT encoder, transformer decoders with self-attention and cross-attention layers, and an MLP regression head to generate pointmaps for consecutive frame images. Crucially, the sharing of information between the two image branches facilitates the correct alignment of pointmaps. The network is trained by minimizing the Euclidean distance between the predicted pointmaps and the actual points:

$$L_{\text{reg}} = \sum_{v=(1,2)} \sum_{i \in D} \left\| \frac{1}{z} X_i^v - \frac{1}{z} \bar{X}_i^v \right\|, \quad (1)$$

where $D \subseteq \{1 \dots W\} \times \{1 \dots H\}$ and z is a scale normalization factor, calculated as $z = \frac{1}{2|D|} \sum_{v=(1,2)} \sum_{i \in D} \|X_i^v\|$.

Although the application of pointmaps might seem counterintuitive, it enables effective representation of 3D shapes in image space and allows for triangulation between rays from different viewpoints without being limited by the quality of depth estimation [18]. Subsequently, we employ the robust and well-established RANSAC [36] with PnP [37] to

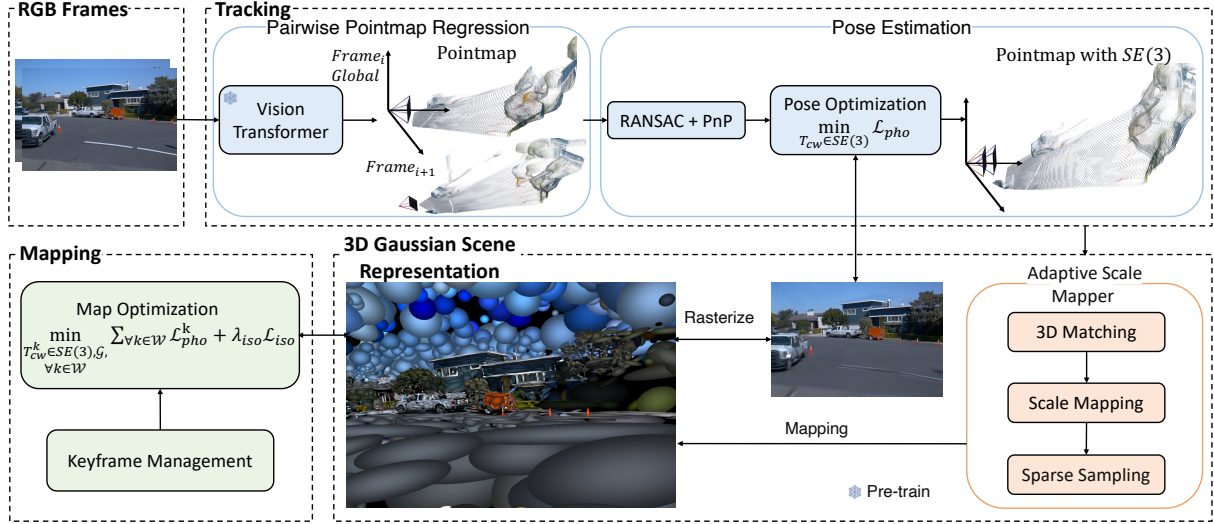


Fig. 1. **SLAM System Pipeline:** Each frame inputs an RGB image for tracking. The current and previous frames are input as a pair into the Pointmap Regression network for pose estimation, followed by pose optimization based on the current Gaussian map. At keyframes, mapping is performed and the pointmap is processed by the Adaptive Scale Mapper for new Gaussian mapping. Camera pose and Gaussian map are jointly optimized in the local window.

infer the relative pose T_{trans}^k between the two frames. Using this method, we calculate the pose for the k -th frame as $T^k = T_{trans}^k T^{k-1}$.

2) **Pose Optimization:** To achieve precise camera pose tracking, we aim to establish a system where the photometric loss is differentiable with respect to the pose, calculated from the rendered image generated using the estimated pose. The camera poses T is described as rotations and translations in 3D space and is represented by the special Euclidean group $SE(3)$, which is a manifold with nonlinear group structure. Since the Lie group $SE(3)$ is not closed under addition [38], it complicates the use of gradient-based methods for optimization. To address this, we linearize $SE(3)$ into its corresponding Lie algebra $\mathfrak{se}(3)$, allowing the use of standard gradient descent techniques for optimization.

This linearization is achieved via the exponential mapping $\exp(\xi)$, where $\xi = (\omega, \nu)$ represents the infinitesimal generators of rotation and translation in the Lie algebra. The Jacobian matrix derived from the Lie algebra allows us to ensure the differentiability of the camera pose T_{CW} in the photometric loss function L_{pho} , and to eliminate redundant computations [14]. Using the chain rule, we first compute the derivatives of the 2D Gaussians $\mathcal{N}(\mu_I, \Sigma_I)$ with respect to the camera pose T_{CW} . These 2D Gaussians are obtained by applying EWA splatting [39] to the 3D Gaussians $\mathcal{N}(\mu_W, \Sigma_W)$. The derivatives of μ_I and Σ_I are given by [3].

$$\frac{\partial \mu_I}{\partial T_{CW}} = \frac{\partial \mu_I}{\partial \mu_C} \cdot [I - \mu_c^\times], \quad (2)$$

$$\frac{\partial \Sigma_I}{\partial T_{CW}} = \frac{\partial \Sigma_I}{\partial J} \cdot \frac{\partial J}{\partial \mu_C} \cdot [I - \mu_c^\times] + \frac{\partial \Sigma_I}{\partial W} \cdot \begin{bmatrix} 0 & -W_{:,1}^\times \\ 0 & -W_{:,2}^\times \\ 0 & -W_{:,3}^\times \end{bmatrix}, \quad (3)$$

where $^\times$ denotes the skew symmetric matrix of a 3D vector, W is the rotational component of T_{CW} , and $W_{:,i}^\times$ refers to the i th column of the matrix.

This calculation is essential for the differentiability of camera pose. We define the photometric loss L_{pho} as:

$$L_{pho} = \|r(\mathcal{G}, T_{CW}) - \bar{I}\|_1. \quad (4)$$

where r denotes per-pixel differentiable rendering function, producing the image through Gaussians $\mathcal{G}$ and camera pose T_{CW} , and $\bar{I}$ represents the ground truth image.

Finally, the derivative of the photometric loss L_{pho} with respect to the pose T_{CW} can be computed using the chain rule:

$$\nabla_T L_{pho} = \frac{\partial L_{pho}}{\partial r} \cdot \left(\frac{\partial r}{\partial \mu_I} \cdot \frac{\partial \mu_I}{\partial T_{CW}} + \frac{\partial r}{\partial \Sigma_I} \cdot \frac{\partial \Sigma_I}{\partial T_{CW}} \right) \quad (5)$$

By following these steps, we link incremental pose updates through the differential of the rendering function to the photometric loss. This enables end-to-end optimization of the camera pose based on 3DGS rendering results, ensuring both high precision and robust pose tracking.

C. 3D Gaussian Scene Representation

1) **3D Gaussian Map:** To achieve real-time rendering while preserving the advantages of volumetric scene representations, our SLAM system employs a 3DGS-based scene representation method [14]. This technique not only enhances rendering speeds but also maintains high flexibility and precision.

In this representation, the scene is modeled using a set of Gaussians centered at the point μ , with the shape and orientation of each Gaussian described by its covariance matrix Σ . The distribution of each Gaussian is defined as [14]:

$$\mathcal{G}(x) = e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)}, \quad (6)$$

where x represents an arbitrary position within the 3D scene.

The covariance matrix is ingeniously decomposed into a scaling matrix S and a rotation matrix R , enhancing control over the scene's geometry [14]:

$$\Sigma = R S S^T R^T. \quad (7)$$

We omit spherical harmonics representing view-dependent radiance. By projecting the 3D Gaussians onto a 2D plane and using tile-based rasterization techniques for efficient sorting and blending, we achieve rapid and precise color rendering. The color of a pixel x' is determined by [14]:

$$C(x') = \sum_{i \in N} c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j), \quad (8)$$

where N is the set of Gaussians influencing pixel x' .

Through this method, our SLAM system can swiftly adapt to dynamic environments, optimizing all Gaussian parameters, including position, rotation, scale, opacity, and color. The entire process is differentiable, facilitating straightforward adjustments and improvements.

2) *Adaptive Scale Mapper*: We utilize the pointmap previously obtained, as referenced in III-B.1, to map the 3D Gaussian map. While the pointmap maintains consistency through shared information across branches, we propose an adaptive scale mapper to enhance robustness against inaccuracies in pointmap regression. This mapper is designed to perform scale-mapping on the pointmaps before inserting new Gaussians based on the pointmaps by performing 3D matching across consecutive frames. Specifically, we match pointmaps $\{X^{k-1}, X^k\}$ and $\{X'^k, X'^{k+1}\}$ generated from cross-view images of three frames ($k-1, k, k+1$), allowing us to measure the relative distances of the same points across frames. To quantify the scale changes between consecutive frames, we define the scale ratio ρ_{ij} as:

$$\rho_{ij} = \frac{\|X_i'^k - X_j'^{k+1}\|}{\|X_i^{k-1} - X_j^k\|} \quad (9)$$

where X_i^{k-1} and X_j^k are corresponding points in frame k , and $X_i'^k$ and $X_j'^{k+1}$ are their counterparts in frame $k+1$. This ratio reflects the scale change from frame k to $k+1$.

By averaging multiple ρ_{ij} values, we estimate the overall scene's mean scale change. To maintain scale consistency throughout the sequence, we use the first frame as a reference and calculate the scale factor for each frame relative to the first by cumulatively multiplying the average scale ratios:

$$S_k = S_{k-1} \cdot \frac{1}{N} \sum_{ij} \frac{\|X_i'^k - X_j'^{k+1}\|}{\|X_i^{k-1} - X_j^k\|} \quad (10)$$

This approach ensures frame-to-frame scale consistency, enabling the scale factors to be used to map subsequent frame pointmap coordinates, crucial for precise 3D mapping and location tracking in outdoor scenes. Finally, based on our observations, not all 3D Gaussian points contribute to mapping; therefore, we introduce a sparse subsampling method. This method employs a hierarchical structure to effectively control the number of 3D Gaussian points, ensuring high-quality mapping while reducing processing time.

D. Mapping

1) *Keyframe Management*: A good keyframe selection strategy should ensure sufficient viewpoint overlap while

avoiding redundant keyframes. Since it is infeasible to jointly optimize the Gaussian scene and camera pose with all keyframes, we manage a local keyframe window $\mathcal{W}$ to select non-redundant keyframes observing the same area, providing better multi-view constraints for subsequent mapping optimization. With this in mind, we adopt the keyframe management strategy from [3]: selecting keyframes based on covisibility and managing the local window by evaluating the overlap with the most recent keyframe.

2) *Gaussian Map Optimization*: At each keyframe, we optimize the Gaussian map by jointly optimizing the Gaussian attributes and camera poses within the currently managed local keyframe window $\mathcal{W}$, performing local window BA. The optimization is still carried out by minimizing photometric loss. To reduce excessive stretching of the ellipsoids, we employed isotropic regularization [3]:

$$L_{\text{iso}} = \sum_{i=1}^{|\mathcal{G}|} \|s_i - \tilde{s}_i \cdot \mathbf{1}\|_1 \quad (11)$$

to penalise the scaling parameters s_i . The mapping optimization task can be summarized as:

$$\min_{\substack{T_{CW}^k \in SE(3), \mathcal{G} \\ \forall k \in \mathcal{W}}} \sum_{\forall k \in \mathcal{W}} L_{\text{pho}}^k + \lambda_{\text{iso}} L_{\text{iso}}. \quad (12)$$

3) *Adaptive Learning Rate Adjustment*: In classical indoor SLAM datasets, the camera typically captures a small scene in a loop, causing the learning rate for Gaussian optimization to gradually decay as the number of cumulative iterations increases. However, the outdoor data we are studying is captured by a front-facing vehicle camera in street scenes, where the traversed areas are not revisited. Therefore, a different learning rate decay strategy is required. We aim for the learning rate to gradually decay when the vehicle is traveling along straight roads, and to increase when the vehicle encounters slope or makes turns in order to optimize new scenes. To address this, we propose an adaptive learning rate adjustment based on rotation angle. We still adjust the learning rate based on cumulative iterations N_{iter} , and then adaptively adjust the cumulative iterations. Suppose the current keyframe and the last keyframe have rotation matrices R_1, R_0 respectively, the relative rotation matrix is $R_{\text{diff}} = R_0^T R_1$, from which the rotation radian can be computed:

$$\theta_{\text{rad}} = \cos^{-1} \left(\frac{\text{trace}(\mathbf{R}_{\text{diff}}) - 1}{2} \right). \quad (13)$$

We then convert θ_{rad} to degrees θ . If $\theta > 2$, we adjust the cumulative iterations:

$$N_{\text{iter}}^{\text{new}} = N_{\text{iter}} \times \left(1 - \sqrt{\frac{\theta}{90}} \right). \quad (14)$$

When the rotation reaches 90 degrees, iterations are reset. The square root adjustment ensures that small angle changes lead to a more significant increase in learning rate. The adaptive learning rate adjustment effectively improved the quality of mapping in the later stages, with detailed analysis provided in the ablation study section.

TABLE I

TRACKING AND RENDERING RESULTS ON 9 WAYMO SEGMENTS. ATE RMSE[M] FOR TRACKING, PSNR, SSIM, AND LPIPS FOR RENDERING. BEST RESULTS ARE HIGHLIGHTED AS **FIRST**, **SECOND**.

Segment	Metric	Method				
		NICER-SLAM	GORIE-SLAM	Photo-SLAM	MonoGS	Ours
100613	ATE↓	19.39	0.302	14.28	6.953	0.324
	PSNR↑	11.46	18.58	14.29	21.89	24.41
	SSIM↑	0.624	0.750	0.655	0.779	0.811
	LPIPS↓	0.705	0.595	0.794	0.543	0.360
13476	ATE↓	8.18	0.569	25.85	3.366	0.422
	PSNR↑	8.59	15.06	17.36	20.95	22.29
	SSIM↑	0.507	0.533	0.726	0.723	0.733
	LPIPS↓	0.817	0.737	0.663	0.693	0.602
106762	ATE↓	35.59	0.405	58.32	18.16	0.893
	PSNR↑	10.46	20.60	18.95	22.24	26.19
	SSIM↑	0.425	0.770	0.802	0.814	0.851
	LPIPS↓	0.670	0.507	0.558	0.515	0.326
132384	ATE↓	25.22	0.142	3.752	12.08	0.436
	PSNR↑	15.12	20.69	20.03	23.48	26.98
	SSIM↑	0.782	0.790	0.839	0.856	0.883
	LPIPS↓	0.536	0.453	0.510	0.427	0.283
152706	ATE↓	18.67	0.425	18.10	9.180	0.309
	PSNR↑	11.55	17.87	17.92	22.52	23.95
	SSIM↑	0.625	0.626	0.766	0.791	0.802
	LPIPS↓	0.745	0.677	0.768	0.649	0.533
153495	ATE↓	15.42	1.202	6.407	5.718	1.576
	PSNR↑	11.15	19.40	18.21	21.49	23.66
	SSIM↑	0.487	0.726	0.730	0.782	0.800
	LPIPS↓	0.743	0.568	0.746	0.635	0.499
158686	ATE↓	20.59	0.589	21.99	8.396	1.076
	PSNR↑	12.65	18.93	16.96	21.25	21.71
	SSIM↑	0.609	0.694	0.696	0.734	0.731
	LPIPS↓	0.756	0.539	0.684	0.574	0.468
163453	ATE↓	22.68	0.646	25.39	11.21	1.719
	PSNR↑	15.38	19.01	18.58	19.28	21.00
	SSIM↑	0.690	0.732	0.739	0.743	0.745
	LPIPS↓	0.748	0.525	0.694	0.642	0.506
405841	ATE↓	10.60	0.546	5.466	1.703	0.800
	PSNR↑	13.66	19.32	17.31	23.14	25.72
	SSIM↑	0.621	0.698	0.724	0.804	0.840
	LPIPS↓	0.815	0.553	0.655	0.522	0.333
Avg.	ATE↓	19.59	0.536	19.95	8.529	0.839
	PSNR↑	12.22	18.83	17.73	21.80	23.99
	SSIM↑	0.622	0.702	0.741	0.780	0.800
	LPIPS↓	0.726	0.572	0.674	0.577	0.434

IV. EXPERIMENTS

A. Implementation and Experiment Setup

1) *Datasets*: We evaluate the Waymo open dataset [40], focusing on tracking accuracy, novel view rendering performance comparison, and ablation study. Waymo dataset, collected by autonomous vehicles, contains outdoor street scenes. We use the front-facing RGB images captured by the vehicle’s cameras as input.

2) *Baseline Methods*: We compare our method with four SLAM approaches that support monocular RGB-only input and novel view rendering, including NICER-SLAM [1], GORIE-SLAM [30], Photo-SLAM [16], and MonoGS [3].

3) *Metrics*: To assess novel view rendering performance, we use PSNR, SSIM [41], and LPIPS metrics, calculated on frames excluding keyframes (i.e., training frames). For tracking accuracy, we use ATE RMSE (in meters) as the evaluation metric.

4) *Implementation Details*: We run our SLAM on a single NVIDIA RTX A6000 GPU. As with MonoGS, rasterization and gradients computation for Gaussian attributes and cam-

era pose are implemented through CUDA. The rest of the SLAM pipeline is developed using PyTorch. We employed the best-performing pre-trained pointmap regression network [18] in our tests. Local window size $|\mathcal{W}| = 8$, isotropic regularization coefficient $\lambda_{iso} = 10$.

B. Experiment Results

TABLE I shows the performance of tracking and novel view rendering. Our method achieves the best novel view rendering performance across all segments. Compared to MonoGS, which also use Gaussian scene representation, our approach improves the average PSNR by 10%. In terms of tracking accuracy, our method slightly lags behind GORIE-SLAM but significantly outperforms the other methods. GORIE-SLAM employs a frame-to-frame tracking module based on optical flow and performs global BA every 20 frames and at the end of the system. In contrast, our tracking relies on Gaussian map-based pose alignment, which requires robust scene reconstruction—a challenging task in unbounded outdoor environments. As shown in Fig. 3, compared to MonoGS, which tracks in a similar manner, our tracking trajectories are noticeably more accurate, with no significant drift, and effectively handle sharp turns, demonstrating the strength of our approach. Additionally, despite not incorporating global BA or backend filtering, we achieve results comparable to GORIE-SLAM and even outperform it on two segments, while also possessing superior NVS capabilities, highlighting the potential of our method. Moreover, implementing efficient and accurate global BA based on Gaussian scenes is no trivial task.

Fig. 2 presents the novel view rendering results, where our method renders high-fidelity images that accurately capture details of vehicles, streets, and buildings in both forward and side views. While GORIE-SLAM achieves the best tracking performance, its rendered images suffer from missing pixels and distortions. Although MonoGS also uses Gaussian representation, its rendered images are very blurry. These results clearly demonstrate the superior novel view rendering capability of our method in unbounded outdoor scenes.

TABLE II

ABLATION STUDY RESULTS: IMPACT ON TRACKING AND NOVEL VIEW RENDERING PERFORMANCE AFTER REMOVING EACH MODULE.

Method	ATE RMSE	PSNR	SSIM	LPIPS
w/o lr adjustment	1.836	23.08	0.781	0.436
w/o scale mapper	1.095	23.49	0.787	0.450
w/o pointmap regression	11.18	18.47	0.734	0.614
Ours	0.839	23.99	0.800	0.434

C. Ablation Study

In this section, we demonstrate the importance of pointmap regression to the overall SLAM framework, as well as the impact of adaptive scale mapper and adaptive learning rate adjustment on performance. The average results across the 9 Waymo segments are reported in TABLE II.



Fig. 2. **Novel View Rendering Results on 4 Waymo segments.** For unbounded outdoor scenes, our method renders high-fidelity images, accurately capturing details of vehicles, streets, and buildings. In contrast, MonoGS and GIORIE-SLAM exhibit rendering distortions and blurriness.

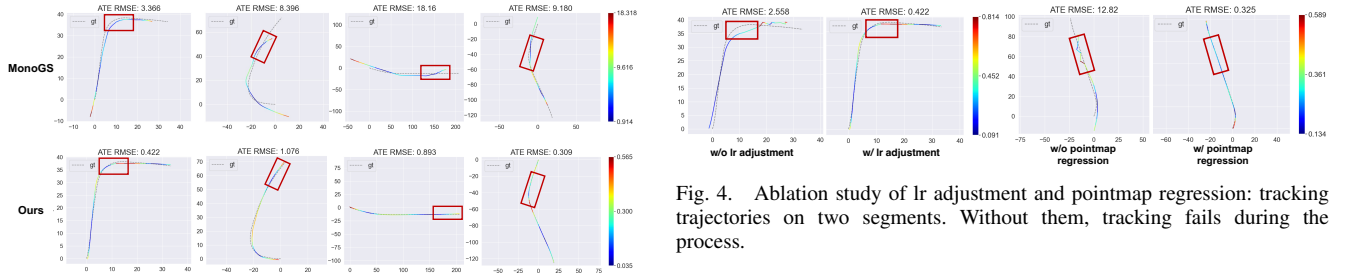


Fig. 3. Comparison of tracking trajectories with MonoGS on 4 segments. Our method greatly enhances tracking accuracy, with no noticeable drift.

1) *Adaptive Learning Rate Adjustment:* Learning rate adjustment is crucial for tracking accuracy, particularly during turns. Fig. 4 shows that significant trajectory drift occurs during turns without learning rate adjustment, affecting subsequent tracking. This is because after a turn, the Gaussian map for the new viewpoint requires a higher learning rate for proper adjustment, and our tracking relies on an accurate Gaussian map.

2) *Adaptive Scale Mapper:* Without adaptive scale mapper, both tracking and novel view rendering performance degrade. We have learned that pointmaps regressed from different frames have scale discrepancies, and failure to adjust these will misplace newly inserted Gaussians, negatively impacting the entire SLAM system’s performance.

3) *Pairwise Pointmap Regression:* Without pointmap regression, we use the estimated pose from the previous frame as the initialization and generate depth maps through depth rasterization with added noise for Gaussian insertion. This approach produces poor results, highlighting the importance

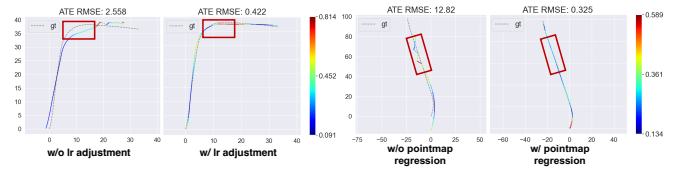


Fig. 4. Ablation study of lr adjustment and pointmap regression: tracking trajectories on two segments. Without them, tracking fails during the process.

of pointmap regression, as its pre-trained information is crucial for accurate outdoor scene reconstruction.

V. CONCLUSIONS

In this paper, we introduce OpenGS-SLAM, an RGB-only SLAM system based on 3DGS representation for unbounded outdoor scenes. Our approach integrates a pointmap regression network with 3DGS representation, ensuring precise camera pose tracking and excellent novel view synthesis capabilities. Compared to other 3DGS-based SLAM systems, our method offers superior tracking accuracy and robustness in outdoor settings, making it highly practical for real-world applications.

ACKNOWLEDGMENT

This research is supported by the National Natural Science Foundation of China (No. 62406267), Guangzhou-HKUST(GZ) Joint Funding Program (Grant No. 2025A03J3956), the Guangzhou Municipal Science and Technology Project (No. 2025A04J4070), the Guangzhou Municipal Education Project (No. 2024312122) and Education Bureau of Guangzhou Municipality.

REFERENCES

- [1] Z. Zhu, S. Peng, V. Larsson, Z. Cui, M. R. Oswald, A. Geiger, and M. Pollefeys, “Nicer-slam: Neural implicit scene encoding for rgb slam,” in *2024 International Conference on 3D Vision (3DV)*. IEEE, 2024, pp. 42–52.
- [2] J. Ren, H. Wu, H. Xiong, and H. Wang, “Sca3d: Enhancing cross-modal 3d retrieval via 3d shape and caption paired data augmentation,” *arXiv preprint arXiv:2502.19128*, 2025.
- [3] H. Matsuki, R. Murai, P. H. Kelly, and A. J. Davison, “Gaussian splatting slam,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 18 039–18 048.
- [4] E. Sandström, K. Tateno, M. Oechsle, M. Niemeyer, L. Van Gool, M. R. Oswald, and F. Tombari, “Splat-slam: Globally optimized rgb-only slam with 3d gaussians,” *arXiv preprint arXiv:2405.16544*, 2024.
- [5] R. A. Newcombe, S. Izadi, O. Hilliges, D. Molyneaux, D. Kim, A. J. Davison, P. Kohi, J. Shotton, S. Hodges, and A. Fitzgibbon, “Kinectfusion: Real-time dense surface mapping and tracking,” in *2011 10th IEEE International Symposium on Mixed and Augmented Reality*, 2011, pp. 127–136.
- [6] T. Schops, T. Sattler, and M. Pollefeys, “Bad slam: Bundle adjusted direct rgb-d slam,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.
- [7] T. Whelan, S. Leutenegger, R. Moreno, B. Glocker, and A. Davison, “Elasticfusion: Dense slam without a pose graph,” 07 2015.
- [8] L. Yen-Chen, P. Florence, J. T. Barron, A. Rodriguez, P. Isola, and T.-Y. Lin, “inerf: Inverting neural radiance fields for pose estimation,” in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021, pp. 1323–1330.
- [9] E. Sucar, S. Liu, J. Ortiz, and A. J. Davison, “imap: Implicit mapping and positioning in real-time,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 6229–6238.
- [10] Z. Zhu, S. Peng, V. Larsson, W. Xu, H. Bao, Z. Cui, M. R. Oswald, and M. Pollefeys, “Nice-slam: Neural implicit scalable encoding for slam,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 12 786–12 796.
- [11] X. Yang, H. Li, H. Zhai, Y. Ming, Y. Liu, and G. Zhang, “Vox-fusion: Dense tracking and mapping with voxel-based neural implicit representation,” in *2022 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*. IEEE, 2022, pp. 499–507.
- [12] M. M. Johari, C. Carta, and F. Fleuret, “Eslam: Efficient dense slam system based on hybrid representation of signed distance fields,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 17 408–17 419.
- [13] Y. Zhang, F. Tosi, S. Mattoccia, and M. Poggi, “Go-slam: Global optimization for consistent 3d instant reconstruction,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 3727–3737.
- [14] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3d gaussian splatting for real-time radiance field rendering,” *ACM Trans. Graph.*, vol. 42, no. 4, pp. 139–1, 2023.
- [15] N. Keetha, J. Karhade, K. M. Jatavallabhula, G. Yang, S. Scherer, D. Ramanan, and J. Luiten, “Splatam: Splat track & map 3d gaussians for dense rgb-d slam,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 21 357–21 366.
- [16] H. Huang, L. Li, H. Cheng, and S.-K. Yeung, “Photo-slam: Real-time simultaneous localization and photorealistic mapping for monocular stereo and rgb-d cameras,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 21 584–21 593.
- [17] C. Yan, D. Qu, D. Xu, B. Zhao, Z. Wang, D. Wang, and X. Li, “Gs-slam: Dense visual slam with 3d gaussian splatting,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 19 595–19 604.
- [18] S. Wang, V. Leroy, Y. Cabon, B. Chidlovskii, and J. Revaud, “Dust3r: Geometric 3d vision made easy,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 20 697–20 709.
- [19] W. Zhang, T. Sun, S. Wang, Q. Cheng, and N. Haala, “Hi-slam: Monocular real-time dense mapping with hybrid implicit fields,” *IEEE Robotics and Automation Letters*, 2023.
- [20] L. Liso, E. Sandström, V. Yugay, L. Van Gool, and M. R. Oswald, “Loopy-slam: Dense neural slam with loop closures,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 20 363–20 373.
- [21] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. Montiel, and J. D. Tardós, “Orb-slam3: An accurate open-source library for visual, visual-inertial, and multimap slam,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [22] V. Yugay, Y. Li, T. Gevers, and M. R. Oswald, “Gaussian-slam: Photo-realistic dense slam with gaussian splatting,” *arXiv preprint arXiv:2312.10070*, 2023.
- [23] J. Hu, X. Chen, B. Feng, G. Li, L. Yang, H. Bao, G. Zhang, and Z. Cui, “Cg-slam: Efficient dense rgb-d slam in a consistent uncertainty-aware 3d gaussian field,” *arXiv preprint arXiv:2403.16095*, 2024.
- [24] A. Rosinol, J. J. Leonard, and L. Carlone, “Nerf-slam: Real-time dense monocular slam with neural radiance fields,” in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 3437–3444.
- [25] C.-M. Chung, Y.-C. Tseng, Y.-C. Hsu, X.-Q. Shi, Y.-H. Hua, J.-F. Yeh, W.-C. Chen, Y.-T. Chen, and W. H. Hsu, “Orbeez-slam: A real-time monocular visual slam with orb features and nerf-realized mapping,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 9400–9406.
- [26] Z. Teed and J. Deng, “Droid-slam: Deep visual slam for monocular, stereo, and rgb-d cameras,” *Advances in neural information processing systems*, vol. 34, pp. 16 558–16 569, 2021.
- [27] R. Mur-Artal and J. D. Tardós, “Orb-slam2: An open-source slam system for monocular, stereo, and rgb-d cameras,” *IEEE transactions on robotics*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [28] T. Müller, A. Evans, C. Schied, and A. Keller, “Instant neural graphics primitives with a multiresolution hash encoding,” *ACM transactions on graphics (TOG)*, vol. 41, no. 4, pp. 1–15, 2022.
- [29] H. Li, X. Gu, W. Yuan, L. Yang, Z. Dong, and P. Tan, “Dense rgb slam with neural implicit maps,” in *Proceedings of the International Conference on Learning Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=QUK1ExlbbA>
- [30] G. Zhang, E. Sandström, Y. Zhang, M. Patel, L. Van Gool, and M. R. Oswald, “Glorie-slam: Globally optimized rgb-only implicit encoding point cloud slam,” *arXiv preprint arXiv:2403.19549*, 2024.
- [31] X. Guo, P. Han, W. Zhang, and H. Chen, “Motiongs: Compact gaussian splatting slam by motion filter,” *arXiv preprint arXiv:2405.11129*, 2024.
- [32] P. Zhu, Y. Zhuang, B. Chen, L. Li, C. Wu, and Z. Liu, “Mgs-slam: Monocular sparse tracking and gaussian mapping with depth smooth regularization,” *arXiv preprint arXiv:2405.06241*, 2024.
- [33] Z. Teed, L. Lipson, and J. Deng, “Deep patch visual odometry,” *Advances in Neural Information Processing Systems*, 2023.
- [34] O. Wiles, G. Gkioxari, R. Szeliski, and J. Johnson, “Synsin: End-to-end view synthesis from a single image,” 2020. [Online]. Available: <https://arxiv.org/abs/1912.08804>
- [35] J. Wang, B. Sun, and Y. Lu, “Mypnet: Multi-view point regression networks for 3d object reconstruction from a single image,” 2018. [Online]. Available: <https://arxiv.org/abs/1811.09410>
- [36] M. A. Fischler and R. C. Bolles, “Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography,” *Commun. ACM*, vol. 24, no. 6, p. 381–395, jun 1981. [Online]. Available: <https://doi.org/10.1145/358669.358692>
- [37] V. Lepetit, F. Moreno-Noguer, and P. Fua, “Ep n p: An accurate o (n) solution to the p n p problem,” *International journal of computer vision*, vol. 81, pp. 155–166, 2009.
- [38] T. D. Barfoot, *State estimation for robotics*. Cambridge University Press, 2024.
- [39] M. Zwicker, H. Pfister, J. Van Baar, and M. Gross, “Ewa volume splatting,” in *Proceedings Visualization, 2001. VIS’01*. IEEE, 2001, pp. 29–538.
- [40] P. Sun, H. Kretschmar, X. Dotiwalla, A. Chouard, V. Patnaik, P. Tsui, J. Guo, Y. Zhou, Y. Chai, B. Caine, V. Vasudevan, W. Han, J. Ngiam, H. Zhao, A. Timofeev, S. Ettinger, M. Krivokon, A. Gao, A. Joshi, Y. Zhang, J. Shlens, Z. Chen, and D. Anguelov, “Scalability in perception for autonomous driving: Waymo open dataset,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.
- [41] Z. Wang, A. Bovik, H. Sheikh, and E. Simoncelli, “Image quality assessment: from error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.